

UNIVERSITY OF TRANSPORT AND COMMUNICATIONS

FACULTY OF INTERNATIONAL EDUCATION



GRADUATION PROJECT

BUILD A 2D GALAXY SAVIOR GAME

HIGH-QUALITY PROGRAM OF ENGLISH – VIETNAMESE

INFORMATION TECHNOLOGY

2022 - 2026 (Course 63)

by

TRAN TIEN DUNG

HANOI - JUNE 2026

UNIVERSITY OF TRANSPORT AND COMMUNICATIONS

FACULTY OF INTERNATIONAL EDUCATION



GRADUATION PROJECT

BUILD A 2D GALAXY SAVIOR GAME

HIGH-QUALITY PROGRAM OF ENGLISH – VIETNAMESE
INFORMATION TECHNOLOGY

2022 - 2026 (Course 63)

by

TRAN TIEN DUNG

HANOI - JUNE 2026

ACKNOWLEDGEMENTS

Firstly, I would like to express my gratitude to all the teachers in the **FACULTY OF INTERNATIONAL EDUCATION - University of Transport and Communications**, who have devoted their passion and knowledge to us during the past four years at this school. The knowledge that teachers have imparted to us is invaluable and will be our luggage throughout life!

I would like to extend my deepest thanks to my teacher - **Dr. Vu Huan**, who has wholeheartedly helped and guided me throughout the implementation of the project. The valuable guidance from the teacher has helped me to complete the project in the best possible way!

Finally, I would like to dedicate my deepest love to my parents and the whole family, who have cared for, nurtured me to become the person I am today. The family has been a solid foundation for me throughout the past years and will continue to be in the life ahead!

Hanoi,month.....2026

Student

Table of Contents

Contents

ACKNOWLEDGEMENTS.....	0
Table of Contents.....	1
Table of Contents Images	3
INTRODUCTION	1
CHAPTER 1: INTRODUCTION TO THE TOPIC	2
1.1. Problem Statement and Motivation	2
1.2. Objectives of the Project.....	3
1.3. Scope and Limitations.....	4
1.4. Development Methodology	5
1.5. Chapter Conclusion.....	6
CHAPTER 2: THEORETICAL BACKGROUND AND TECHNOLOGY OVERVIEW	7
2.1. Programming Language C# in Unity.....	7
2.2. Unity Game Engine.....	14
CHAPTER 3: GAME DESIGN.....	22
3.1. Game Concept and Story	22
3.2. Gameplay and Core Mechanics	23

3.3. Character and Environment Design.....	26
3.4. System Design and Analysis	26
3.5. User Interface Design	27
3.6. Audio Design	31
3.7. Chapter Conclusion.....	31
CHAPTER 4: IMPLEMENTATION AND ACHIEVED RESULTS.....	33
4.1. Achieved Results	33
4.2. Setup Instructions	33
4.3. Result Evaluation.....	33
CONCLUSION.....	35
REFERENCES	36

Table of Contents Images

Figure 1 Singleton Pattern	10
Figure 2 Object Pool Pattern.....	12
Figure 3 Observer Pattern	14
Figure 4 Unity icon	15
Figure 5 Scene interface in Unity	17
Figure 6 Component interface in Unity	18
Figure 7 Script interface in Unity	18
Figure 8 Prefab interface in Unity	19
Figure 9 Hierarchy interface in Unity.....	19
Figure 10 Inspector interface in Unity.....	20
Figure 11 Assets interface in Unity	20
Figure 12 Package Manager interface in Unity	21
Figure 13 General Use Case Diagram	27
Figure 14 General Activity Diagram	27
Figure 15 Menu scene interface.....	28
Figure 16 Hub scene interface	29
Figure 17 Play scene interface	30

INTRODUCTION

In recent years, the gaming industry has grown rapidly, turning games into immersive experiences that combine storytelling, gameplay, and visuals. Despite the popularity of online games, offline PC games remain important thanks to their stability and strong single-player experiences.

Among game genres, action-adventure games with narrative elements are especially popular, as they challenge players while engaging them through stories, level progression, and boss battles.

With an interest in game development, I chose the project “Building a 2D Action Game: Galaxy Savior.” This is a 2D offline PC game where players act as a space hero exploring planets, defeating enemies, and uncovering a larger mystery.

The game features a progression system with levels and bosses, along with a gradually unfolding storyline that leads to a final cosmic battle. Through this project, I aim to improve my Unity programming skills, game system design, and creativity.

Due to limited experience, the project may still have shortcomings, and I look forward to receiving feedback to further improve it.

CHAPTER 1: INTRODUCTION TO THE TOPIC

1.1. Problem Statement and Motivation

1.1.1. *Motivation for Choosing the Game Genre*

- In recent years, the PC game industry has experienced significant growth, offering a wide variety of game genres and experiences. Although 3D games currently dominate the market, 2D games continue to maintain their popularity due to their simplicity, accessibility, artistic style, and creative gameplay design. Many successful indie games have demonstrated that 2D games can still provide highly engaging and memorable experiences for players.
- Among the various genres, 2D action games—especially top-down shooter and bullet hell games—have attracted a large number of players because of their fast-paced gameplay, challenging mechanics, and intense combat systems. These games require players to develop quick reflexes, precise movement, and strategic thinking while avoiding complex bullet patterns.
- The inspiration for this project comes from classic bullet hell and space shooter games that combine challenging gameplay with exploration and storytelling elements. The goal is not only to recreate the excitement of the genre but also to improve the gameplay experience through smooth controls, progressive difficulty, and immersive level design.

1.1.2. *Entertainment Demand and Market Potential*

- In Vietnam, the number of PC gamers continues to increase, especially among students and young adults. Offline games remain popular because they provide stable performance and do not require a constant Internet connection. This makes offline games convenient, accessible, and suitable for a wide range of players.
- However, the offline PC game market in Vietnam still has significant untapped potential. The number of locally developed games remains relatively limited, particularly in the 2D action genre. Therefore, developing a game project such as

Galaxy Savior can contribute to the growth of the local game development community while also providing entertainment value for players.

1.2. Objectives of the Project

1.2.1. General Objective

The main objective of this graduation project is to design and develop a complete and playable 2D action game called ***Galaxy Savior*** using the Unity game engine.

1.2.2. Specific Objectives

The specific objectives of the project include:

- Developing the core gameplay system, including:
 - Character movement
 - Shooting mechanics
 - Enemy interaction
 - Collision and damage systems
- Designing multiple levels and maps with increasing difficulty.
- Building gameplay systems such as:
 - Character health system
 - Skill and weapon systems
 - Enemy spawning system
 - Boss battle mechanics
- Implementing AI behavior for enemies and bosses to create challenging combat experiences.
- Developing a user-friendly UI/UX system, including:
 - Main menu
 - Health and score display
 - Pause and game over screens

- Creating a simple but engaging storyline that encourages player progression and exploration.
- Improving practical skills in Unity development, game programming, and game system design.

1.3. Scope and Limitations

1.3.1. *Scope of the Project*

- Galaxy Savior is a 2D offline top-down shooter (bullet hell) game developed for the PC platform. The project focuses on creating an action-oriented gameplay experience with exploration and boss-fighting elements.
- The game includes:
- Multiple playable levels
- Different enemy types
 - Boss battles with unique attack patterns
 - Player combat mechanics
 - Basic storyline progression
 - User interface and sound effects
- The project is developed using the Unity game engine and programmed primarily in C#.

1.3.2. *Limitations of the Project*

- Galaxy Savior is a 2D offline top-down shooter (bullet hell) game developed for the PC platform. The project focuses on creating an action-oriented gameplay experience with exploration and boss-fighting elements.
- The game includes:
 - Multiple playable levels
 - Different enemy types

- Boss battles with unique attack patterns
- Player combat mechanics
- Basic storyline progression
- User interface and sound effects
- The project is developed using the Unity game engine and programmed primarily in C#.

1.4. Development Methodology

1.4.1. *Technologies and Tools*

The following technologies and tools are used in the development process:

- Game Engine: Unity
- Programming Language: C#
- Graphics Tools: Photoshop, Aseprite
- Audio Tools: Built-in Unity audio system and external sound resources
- Version Control: Git and GitHub

1.4.2. *Development Process*

- The project follows an iterative and incremental development approach inspired by the Agile methodology.
- The development process includes:
 - Requirement analysis and game concept design
 - Gameplay prototype development
 - Level and enemy design
 - UI/UX implementation
 - Testing and debugging
 - Optimization and final deployment

This approach allows continuous improvement and easier adjustment throughout the development process.

1.5. Chapter Conclusion

This chapter introduced the background, motivation, objectives, scope, and development methodology of the project Galaxy Savior. The chapter also discussed the current situation of the PC game market and the potential of offline 2D games in Vietnam. Based on these analyses, the project aims to create an engaging 2D action game that combines challenging gameplay with an immersive player experience while also improving practical skills in game development using Unity.

CHAPTER 2: THEORETICAL BACKGROUND AND TECHNOLOGY OVERVIEW

2.1. Programming Language C# in Unity

2.1.1. *Introduction to the C# Programming Language in Unity*

C# (C-Sharp) is a modern, object-oriented programming language developed by Microsoft and is the official language used in Unity, a popular game development platform. Unity provides a powerful integrated development environment (IDE) that allows developers to create high-quality 2D and 3D games on various platforms such as PC, consoles, mobile, and web.

C# in Unity brings power and flexibility in coding, helping developers easily create complex and highly interactive games. The combination of Unity and C# allows maximizing the features of this programming language, thereby optimizing the game development process.

2.1.2. *Features of the C# Language in Unity*

- **Object-Oriented:** C# in Unity supports object-oriented programming comprehensively, allowing code organization according to objects and classes, helping to manage and develop projects effectively. Concepts such as inheritance, polymorphism, and encapsulation are clearly and effectively implemented.
- **Good Interaction with Unity API:** Unity provides a rich API to manage objects in the game, handle physics, graphics, sound, and many other features. C# interacts well with the Unity API, allowing developers to easily access and use Unity's features.
- **Simple and Powerful:** C# in Unity is easy to learn and use, eliminating complex and confusing features of other programming languages such as C++.

C# provides modern features such as LINQ (Language Integrated Query), lambda expressions, and async programming, simplifying the game development process.

- **Multitasking and Multithreading:** C# supports multitasking and multithreading programming, allowing games to handle multiple tasks simultaneously, thereby improving performance and user experience. Unity API provides powerful tools to support multithreading programming, helping to manage and coordinate threads effectively.
- **Portability and Platform Independence:** Games developed with Unity and C# can easily be published on various platforms such as Windows, macOS, Linux, iOS, Android, and console platforms. This helps developers reach more players without having to rewrite code for each platform.
- **Rich Library and Tools:** Unity comes with a rich library of tools and assets, helping developers quickly build and deploy games. C# maximizes these tools, from the physics system, lighting system, to user interface design tools.
- **Strong Community Support:** Unity has a large and dynamic community, providing a lot of documentation, tutorials, and useful plugins. C# also has a large community of developers, supporting each other in solving technical problems and sharing knowledge.
- **Security and Privacy:** C# provides strong security features, including encryption, authentication, and access control. Unity also has built-in security mechanisms, helping to protect games from common security threats.

2.1.3. Application in Developing 2D Action Game

In the project “Building a 2D Action Game: Galaxy Savior”, C# is used to implement game logic, handle player interactions, manage game objects, and control various gameplay systems. Specific features include:

- **Object Management:** Game objects such as the player spaceship, enemies, bullets, and bosses are created and managed using C# classes and Unity components.
- **Event Handling:** Player actions such as movement, shooting, taking damage, and interacting with enemies are handled through C# scripts and Unity event systems.
- **Combat and Bullet System:** The game implements a bullet hell mechanism where enemies and bosses generate complex bullet patterns and bullet types. C# is used to control bullet spawning, movement, and collision detection.
- **Physics and Movement:** Using Unity API and C#, object movement and collision handling are implemented to ensure smooth and responsive gameplay.
- **User Interface:** UI elements such as the main menu, health bar, score display, and in-game notifications are designed and managed using Unity UI and C# scripts.

2.1.4. Design Patterns in Game Development

2.1.4.1. Singleton Pattern

The Singleton Pattern is a common design pattern in Unity used to ensure that a class has only one instance throughout the game. It also provides a global access point to that instance.

In Unity, Singleton is commonly used for systems such as:

- Game Manager
- Audio Manager
- UI Manager
- Save System

```
public abstract class StaticInstance<T> : MonoBehaviour where T : MonoBehaviour
{
    public static T Instance { get; private set; }

    protected virtual void Awake()
    {
        if (Instance == null)
            Instance = this as T;
        else
            Destroy(gameObject);
    }

    protected virtual void OnApplicationQuit()
    {
        Instance = null;
        Destroy(gameObject);
    }
}

public abstract class Singleton<T> : StaticInstance<T> where T : MonoBehaviour
{
    protected override void Awake()
    {
        if (Instance != null)
            Destroy(gameObject);

        base.Awake();
    }
}

public abstract class SingletonPersistent<T> : Singleton<T> where T : MonoBehaviour
{
    protected override void Awake()
    {
        base.Awake();
        DontDestroyOnLoad(gameObject);
    }
}
```

Figure 1 Singleton Pattern

2.1.4.2. Object Pool Pattern

The Object Pool Pattern is used to improve game performance by reusing objects instead of continuously creating and destroying them during gameplay. This pattern is especially useful in games with many frequently spawned objects such as bullets, enemies, particle effects, or explosions.

In the *Galaxy Savior* project, the Object Pool Pattern is implemented through the `ObjectPooler` class. The system stores inactive objects inside a queue and reactivates them when needed. This reduces memory allocation and improves runtime performance.

The `SpawnFromPool()` method is used to retrieve objects from the pool, set their position and rotation, and activate them. This pattern is mainly applied to the bullet system and enemy spawning system to ensure smooth gameplay and optimize performance.

```
[System.Serializable]
public class Pool
{
    public string tag;
    public GameObject prefab;
    public int size;
}

public List<Pool> pools;
public Dictionary<string, Queue<GameObject>> poolDictionary;
```

```
public GameObject SpawnFromPool(string tag, Vector3 position, Quaternion rotation)
{
    if (!poolDictionary.ContainsKey(tag))
    {
        Debug.LogWarning("Pool with tag " + tag + " doesn't exist.");
        return null;
    }
    GameObject objectToSpawn = poolDictionary[tag].Dequeue();

    objectToSpawn.SetActive(true);
    objectToSpawn.transform.position = position;
    objectToSpawn.transform.rotation = rotation;

    IPool pooledObj = objectToSpawn.GetComponent<IPool>();

    pooledObj?.OnObjectSpawn();

    poolDictionary[tag].Enqueue(objectToSpawn);

    return objectToSpawn;
}
```

Figure 2 Object Pool Pattern

2.1.4.3. State Pattern

The State Pattern is used to manage different behaviors of an object based on its current state. In game development, this pattern is commonly applied to character AI, enemy behavior, and gameplay systems.

In the *Galaxy Savior* project, the State Pattern is implemented in the enemy AI system through the `EnemyState` enum, which includes three states:

- Roaming
- Chase
- Attack

The enemy changes its behavior depending on the player's position and interaction. When the player is outside the detection range, the enemy moves

randomly in the Roaming state. When the player enters the awareness range, the enemy switches to the Chase state and follows the player. If the enemy gets close enough, it transitions to the Attack state and stops moving to perform an attack. This pattern helps organize enemy behavior clearly, improves code readability, and makes the AI system easier to expand and maintain.

2.1.4.4. Observer Pattern

The Observer Pattern is used to create communication between objects without making them tightly connected. In Unity, this pattern is commonly implemented using events and delegates.

In the *Galaxy Savior* project, the Observer Pattern is applied to the enemy death system. When an enemy dies, the OnEnemyDeath event is triggered and sends information about the enemy's position through EnemyDeathEventArgs.

Other systems, such as the kill count manager, subscribe to this event using OnEnable() and unsubscribe using OnDisable(). When the event is invoked, the IncrementKillCount() method is automatically called to update the player's kill count and UI.

This pattern improves code flexibility, reduces dependencies between systems, and makes the project easier to maintain and expand.

```
public struct EnemyDeathEventArgs
{
    public Vector3 Position;

    public EnemyDeathEventArgs(Vector3 position)
    {
        Position = position;
    }
}

public static event Action<EnemyDeathEventArgs> OnEnemyDeath;
```

```

private void ApplyDamage(float amount)
{
    ShowDamage(amount.ToString());
    currentHealth = Mathf.Clamp(currentHealth - amount, 0, enemyHealthSO.MaxHealth);
    floatingHealthBar.UpdateHealthBar(currentHealth, enemyHealthSO.MaxHealth);
    if (currentHealth <= 0)
    {
        Instantiate(enemyHealthSO.BloodEffect, transform.position, Quaternion.identity);
        Instantiate(enemyHealthSO.BloodSplash, transform.position, Quaternion.identity);
        OnEnemyDeath?.Invoke(new EnemyDeathEventArgs(transform.position));
        gameObject.SetActive(false);
    }
}

private void OnEnable() => EnemyHealth.OnEnemyDeath += IncrementKillCount;

private void OnDisable() => EnemyHealth.OnEnemyDeath -= IncrementKillCount;

private void IncrementKillCount(EnemyHealth.EnemyDeathEventArgs args)
{
    if (KillCount < targetKillCount)
    {
        KillCount++; // Increment the kill count
        UpdateKillCountUI(); // Update the UI
    }
}

```

Figure 3 Observer Pattern

2.2. Unity Game Engine

2.2.1. Concept of Unity



Figure 4 Unity icon

Unity is a cross-platform game development platform, widely used to create electronic games and interactive applications. Unity provides a powerful integrated development environment (IDE), allowing developers to design, develop, and deploy games and interactive applications efficiently. With Unity, developers can easily create high-quality products for various platforms such as PC, consoles, mobile devices, and the web.

Unity is known for its strong support for 2D and 3D graphics, along with a rich set of tools and comprehensive support documentation, simplifying the development process and optimizing the performance of games.

2.2.2. Overview of Unity Engine

2.2.2.1. Features

➤ Graphics and Images:

- Unity Engine provides a wide range of powerful graphics and imagery features, enabling developers to create games and applications with high-quality visuals and vibrant graphic environments.

➤ Physics and Motion:

- Box2D and NVIDIA PhysX Physics Engine: Supports accurate and realistic motion and physics simulation in both 2D and 3D graphics.

- Animation System: Supports creating and managing complex animations, including the animation controller system (Animator Controller) and Blend Trees tools.
- **Sound and Sound Effects:**
 - Sound Management and Processing: Unity provides tools to add sound effects and background music, manage sound in space, and 3D sound effects.
- **Programming and Scripting:**
 - C# Language: Unity uses C# as the main programming language, providing a rich and easy-to-use API to control game logic and interact with objects in the game.
 - Unity Scriptable Objects: Allows data to be stored independently and easily managed.
- **User Interface (UI):**
 - Unity UI: A powerful tool for designing and managing user interfaces, from simple menus to complex UI systems with cross-platform support.
- **Cross-Platform:**
 - Publishing on Multiple Platforms: Unity allows publishing games and applications on various platforms, including Windows, macOS, Linux, iOS, Android, and console platforms like PlayStation and Xbox.
- **Management and Control Tools:**
 - Unity Editor: Integrated development environment, providing tools to design, develop, and test games effectively.
 - Asset Store: An online store that allows developers to buy and sell assets, plugins, and useful tools.
- **Community and Support Documentation:**

- **Large and Dynamic Community:** Unity has a large community, providing a lot of documentation, tutorials, and useful plugins, helping developers solve technical problems and share experiences.

2.2.2.2. *Basic Concepts in Unity*

- **Scene:** Scene is the main organizational unit in Unity. Each scene can be considered a level or a part of the game. The scene contains all objects in the game space, including graphics, sound, light, and game logic.

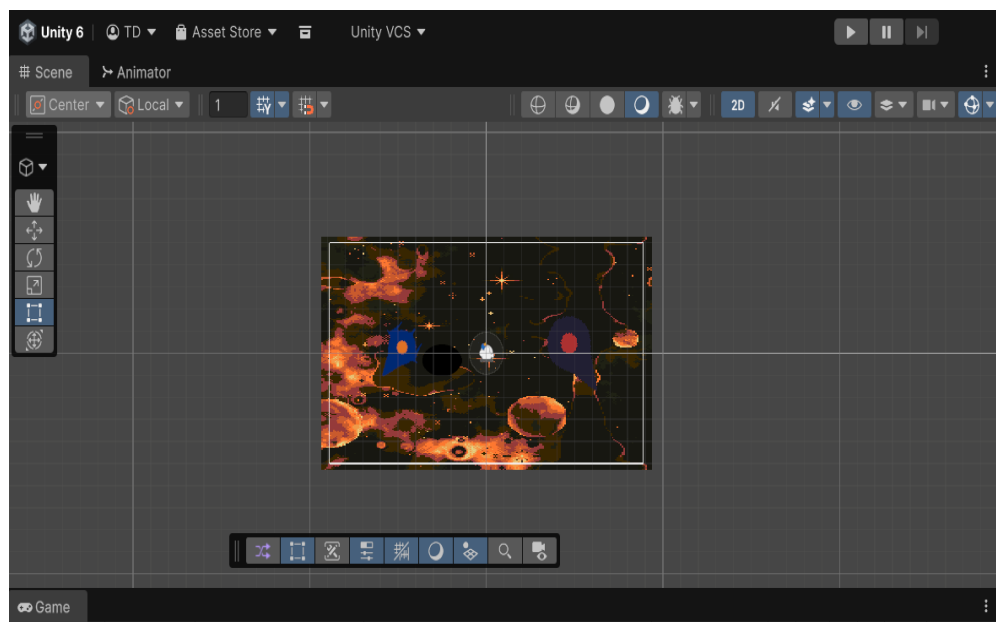


Figure 5 Scene interface in Unity

- **GameObject:** Everything in a scene is a GameObject. GameObjects can be tangible objects such as characters, trees, or intangible objects such as checkpoints, control scripts.
- **Component:** GameObjects are just empty entities until they are assigned Components. Components add functionality to GameObject, such as Mesh Renderer to display graphics, Collider to detect collisions, or Script to add game logic.

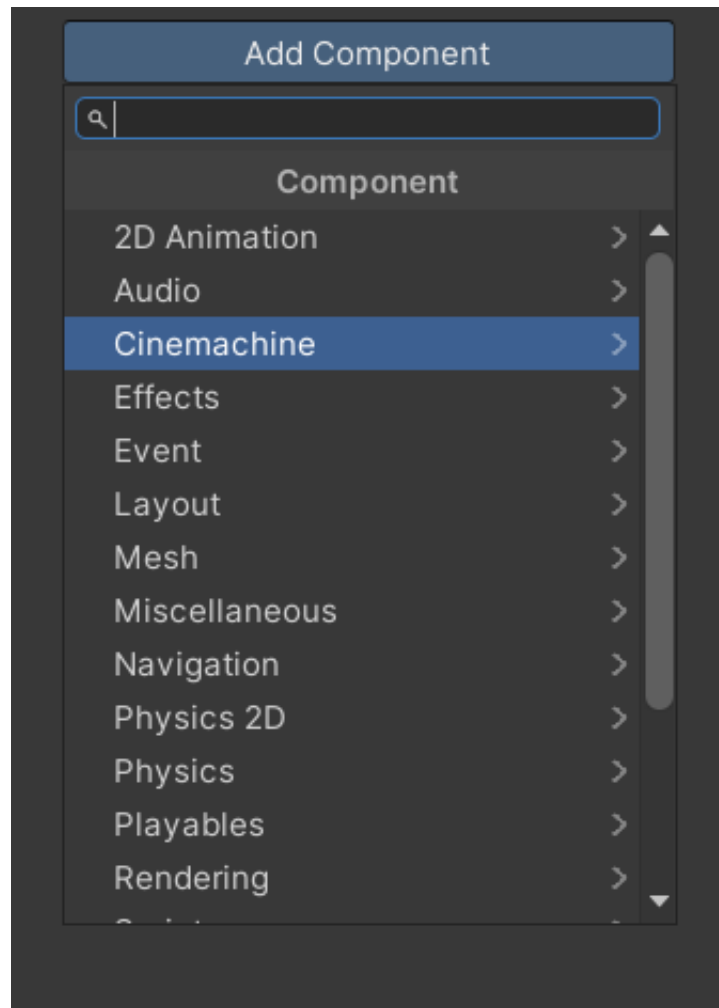


Figure 6 Component interface in Unity

- Script: Scripts are pieces of code written in C#, assigned to GameObjects to control their behavior. Each script is a component and can interact with other components on the same GameObject or other GameObjects.

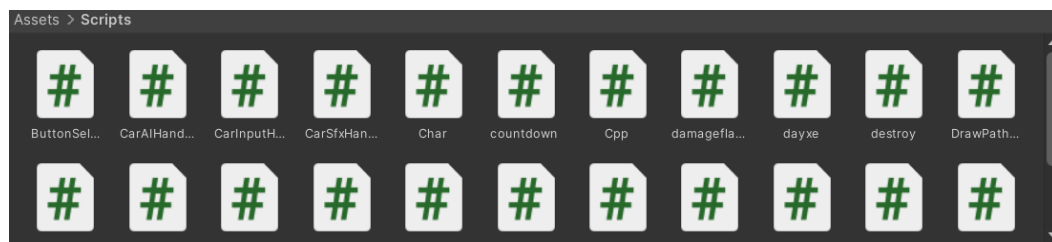


Figure 7 Script interface in Unity

- **Prefab:** Prefabs are GameObjects along with their components stored as templates, making it easy to reuse and manage objects in the game.

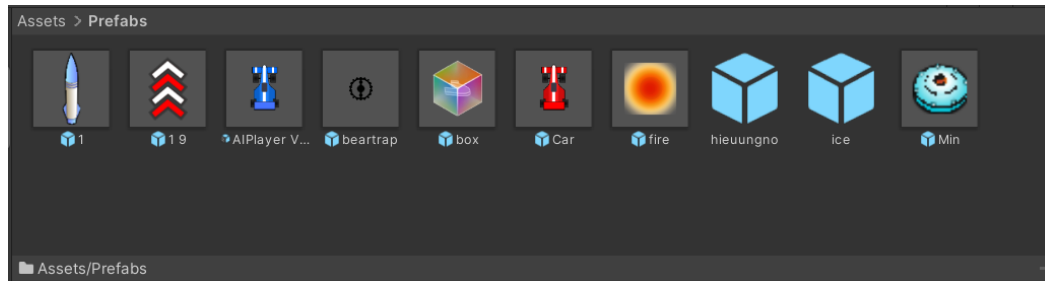


Figure 8 Prefab interface in Unity

- **Hierarchy:** The Hierarchy is where the hierarchical structure of all GameObjects in the scene is displayed. Objects can be organized according to the hierarchy, with child objects dependent on the parent object.

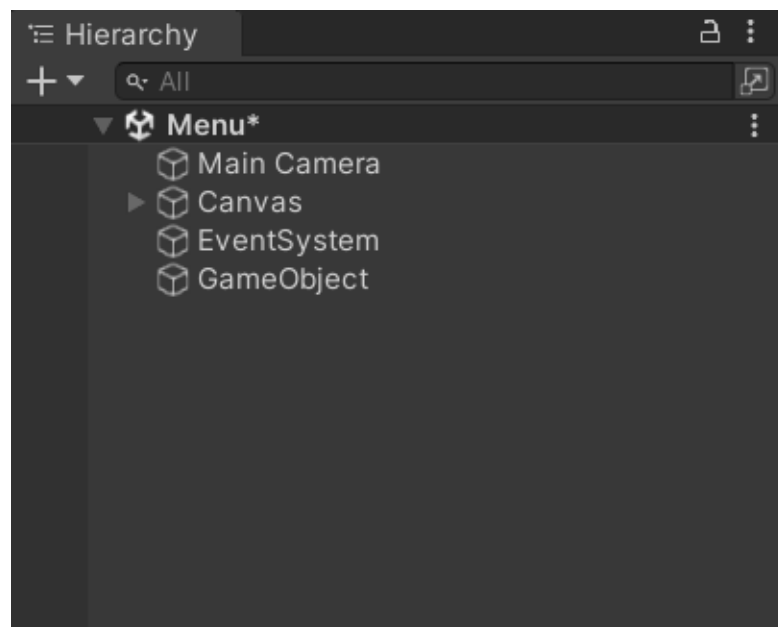


Figure 9 Hierarchy interface in Unity

- **Inspector:** The Inspector is a control panel that allows you to view and edit the properties of GameObjects and their components.

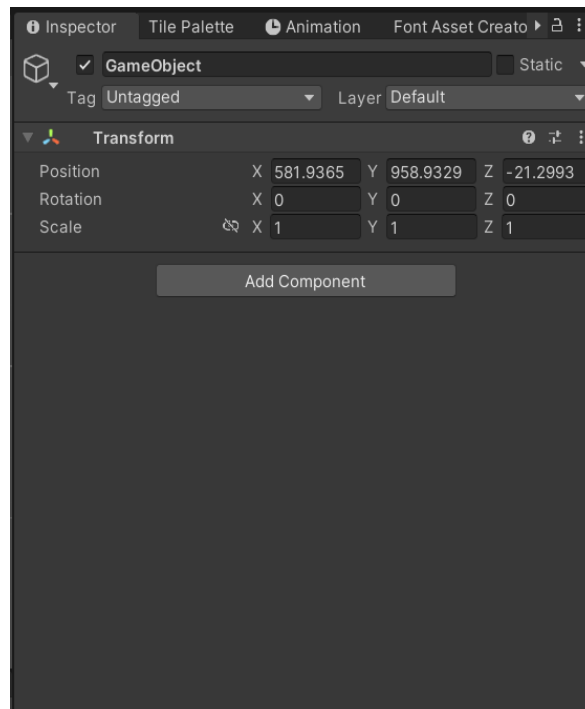


Figure 10 Inspector interface in Unity

- **Assets:** Assets are all files and resources used in the game, including images, sounds, 3D models, scripts, and other resources.

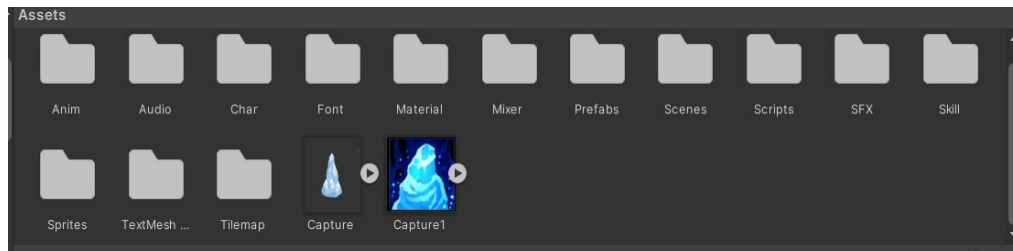


Figure 11 Assets interface in Unity

- **Package Manager:** The Unity Package Manager helps manage additional libraries and tools for the project, allowing you to easily add, delete, and update packages.

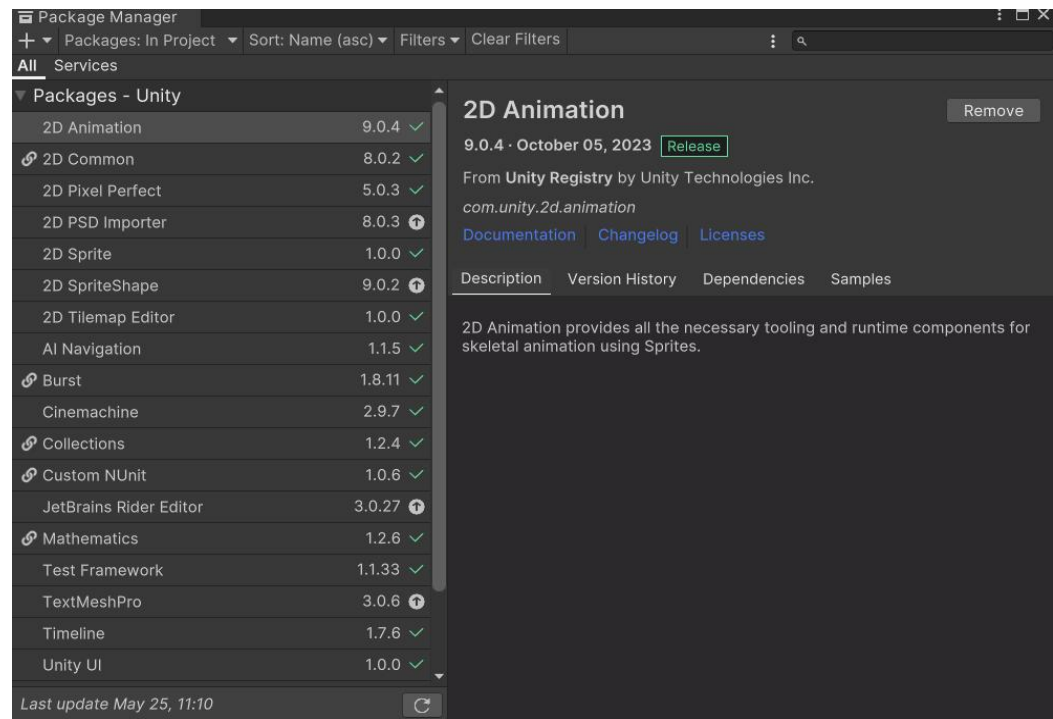


Figure 12 Package Manager interface in Unity

CHAPTER 3: GAME DESIGN

3.1. Game Concept and Story

3.1.1. Introduction

Galaxy Savior is a 2D top-down shooter game developed using the Unity Engine and C# programming language. The game belongs to the bullet hell genre, where players must survive against large numbers of enemy bullets while fighting enemies and bosses in fast-paced combat.

The game combines action gameplay, level progression, and exploration elements to create an engaging player experience. Players control a space hero traveling across different planets to defeat hostile forces and uncover the mystery behind a series of intergalactic disasters threatening the universe.

The project aims to provide smooth gameplay mechanics, responsive controls, challenging enemy encounters, and a flexible progression system that increases replayability.

3.1.2. Game Scenario

The story of *Galaxy Savior* takes place in a futuristic universe where multiple planets are under attack by unknown enemies. These attacks have caused chaos and destruction across the galaxy, threatening the balance of the universe.

The player takes on the role of a space warrior chosen to investigate the source of the disasters and restore peace. During the journey, the player travels through different planets, fights dangerous enemies, and defeats powerful bosses guarding each stage.

As the game progresses, more information about the enemy forces and the hidden truth behind the galactic crisis is gradually revealed through gameplay events and level progression.

3.1.3. Game Objectives

The main objective of the game is to defeat enemies and bosses in order to complete levels and unlock new stages.

Secondary objectives include:

- Surviving increasingly difficult bullet patterns
- Improving player skills and reflexes
- Collecting experience and power-ups
- Upgrading character attributes
- Exploring different planets and gameplay paths

The game uses a non-linear progression system where players can choose different unlocked levels after completing the first stage, increasing freedom and replayability

3.2. Gameplay and Core Mechanics

3.2.1. Core Gameplay

Top-Down Shooter Gameplay

The game uses a top-down camera perspective, allowing players to move freely across the map while attacking enemies from different directions. This perspective provides a clear view of enemy movement and bullet patterns, which is important in bullet hell gameplay.

The player controls a spaceship-like character capable of moving, attacking, dodging bullets, and collecting power-ups during combat.

Bullet Hell Mechanics

One of the core gameplay mechanics in *Galaxy Savior* is the bullet hell system. Enemies and bosses generate large numbers of bullets with different movement patterns and special effects.

The game includes multiple bullet types:

- Normal Bullet
- Poison Bullet
- Freezing Bullet

- Heavy Bullet

Each bullet type affects the player differently and requires different strategies to avoid damage. This system increases gameplay challenge and encourages quick reflexes and accurate movement.

Player Controls and Combat

The player movement system is controlled using the mouse, where the character follows the cursor position. This control method allows accurate movement and improves the player's ability to dodge complex bullet patterns.

Combat is designed to feel smooth and responsive. The player can continuously attack enemies while moving around the battlefield. Different enemy types and bullet patterns force the player to constantly reposition and react quickly.

3.2.2. Enemy and Boss System

Diverse Enemy System

The game contains multiple enemy types with different movement behaviors and attack patterns. Some enemies move directly toward the player, while others attack from a distance or generate unique bullet formations.

Enemy AI is implemented using the State Pattern, allowing enemies to switch between roaming, chasing, and attacking states depending on the player's position.

Boss Battles

Each major stage contains a boss battle. Bosses are significantly stronger than normal enemies and feature unique attack mechanics and complex bullet patterns. Boss battles are designed to test player movement, reaction speed, and pattern recognition. Defeating bosses rewards the player with progression and unlocks new levels.

3.2.3. Progression System

Level Progression

The game is divided into multiple stages located on different planets. The difficulty gradually increases as players progress through the game.

After completing Level 1, players unlock Levels 2, 3, and 4. Completing any two of these levels unlocks the final stage, creating a branching progression system instead of a fixed linear structure.

Open-Ended Storyline

Galaxy Savior features an open-ended progression system that allows players to choose different paths during gameplay. After completing Level 1, players unlock Levels 2, 3, and 4 and can complete them in any order. Completing any two of these levels unlocks the final stage.

This design increases replayability, encourages exploration, and gives players more freedom in experiencing the game's story and challenges.

Upgrade System

Players gain experience points by defeating enemies and collecting experience orbs. When enough experience is collected, the player levels up and receives stat improvements such as:

- Increased health
- Increased attack damage

The amount of experience required increases gradually to maintain gameplay balance.

Power-up System

Map may drop temporary power-ups during gameplay. These power-ups provide advantages such as:

- Increased speed
- Health recovery

This system adds variety and creates dynamic combat situations during gameplay.

Save/Load System

The game uses Unity's ***PlayerPrefs*** system to save player progress, including unlocked levels and completed stages.

This allows players to continue the game without replaying previous levels and supports the game's branching progression design.

3.3. Character and Environment Design

3.3.1. Player Design

The player character is designed as a futuristic space warrior equipped with advanced combat abilities. The character design focuses on visibility and responsive gameplay rather than excessive visual complexity.

3.3.2. Enemy Design

Enemies are designed with different visual appearances and attack styles to create gameplay variety. Each enemy type represents a different combat challenge for the player.

3.3.3. Boss Design

Bosses are visually larger and more detailed than normal enemies. Each boss has unique attack patterns and special mechanics that differentiate boss battles from normal combat encounters.

3.3.4. Art Style and Environment

The game uses a 2D pixel art-style visual design combined with sci-fi environments and space-themed effects. Different planets feature unique backgrounds, lighting, and environmental designs to create visual diversity.

3.4. System Design and Analysis

3.4.1. Development Tools

- Unity Engine for game development
- Visual Studio for C# programming
- Aseprite and online assets for 2D art resources

3.4.2. General Use Case Diagram

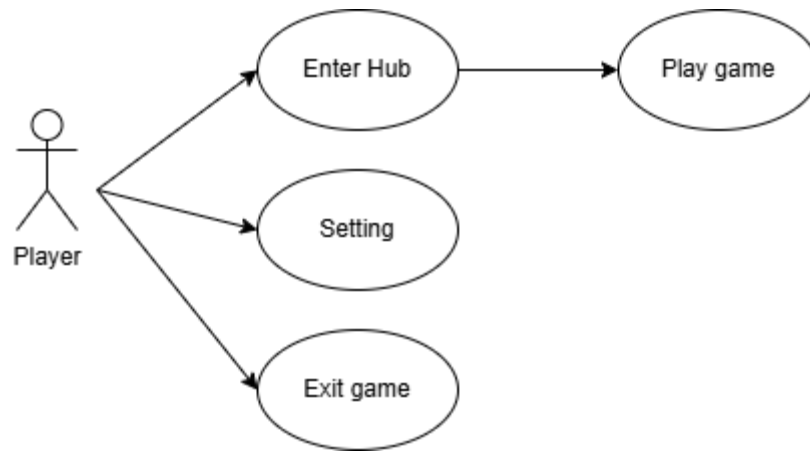


Figure 13 General Use Case Diagram

3.4.3. General Activity Diagram

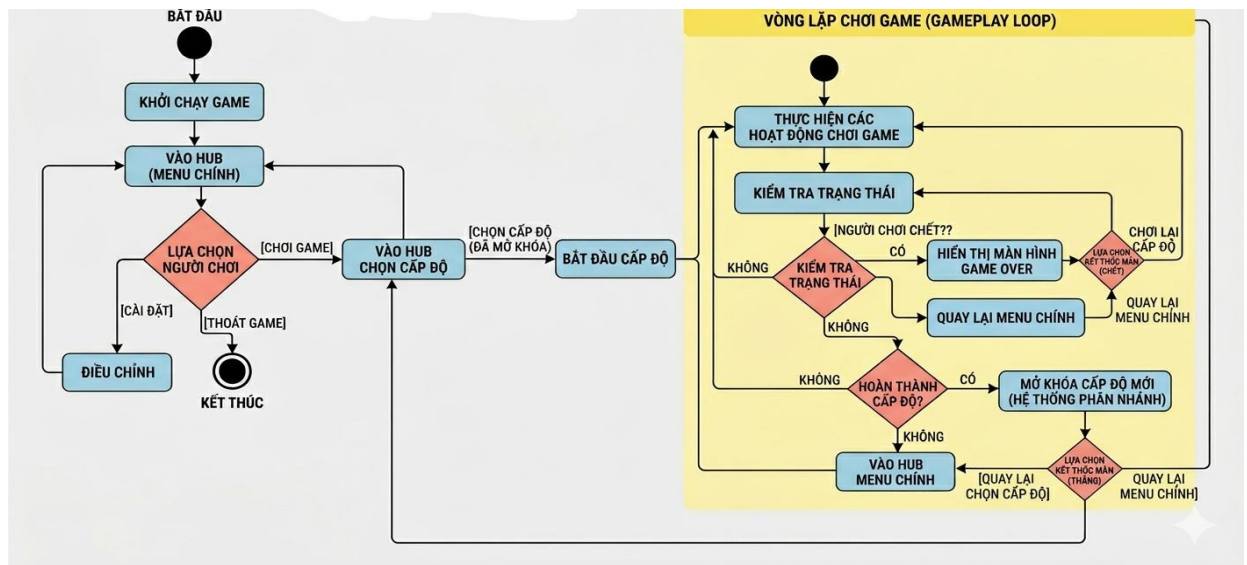


Figure 14 General Activity Diagram

3.5. User Interface Design

3.5.1. Main Menu



Figure 15 Menu scene interface

- There are 3 Buttons:
 - Play: Go to the Hub screen.
 - Settings: Allows players to adjust game configurations such as volume and other in-game settings.
 - About: Displays information about the developer.

3.5.2. Hub Scene

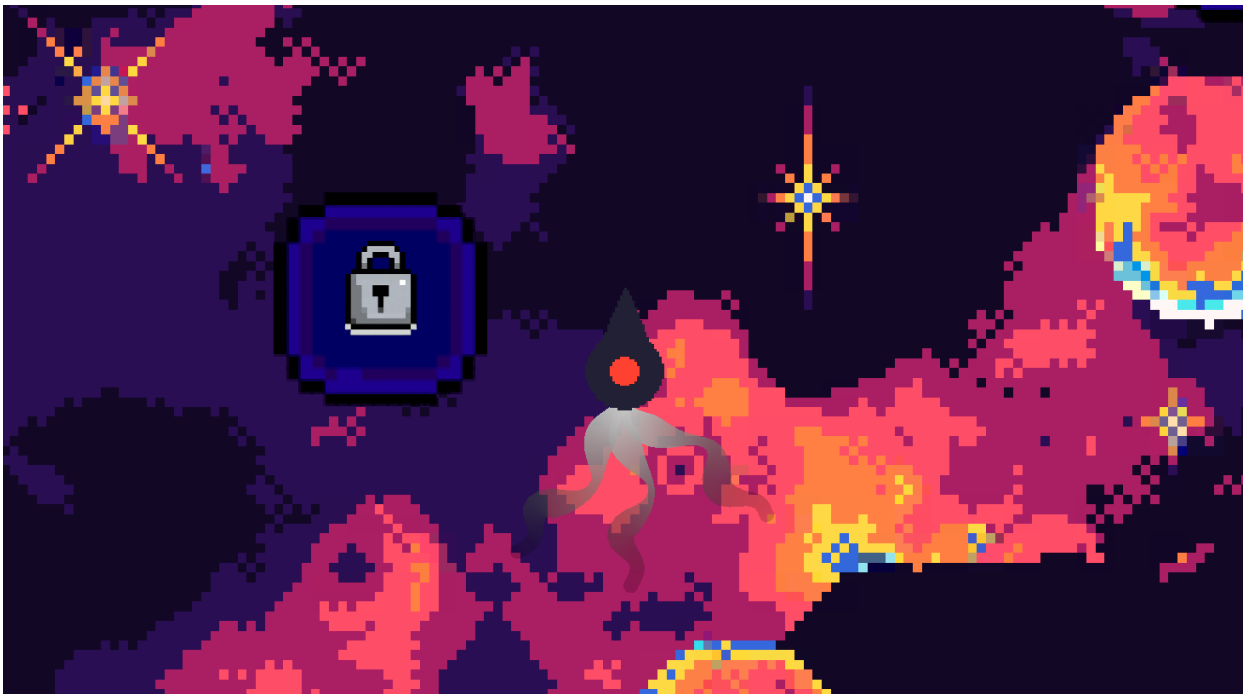
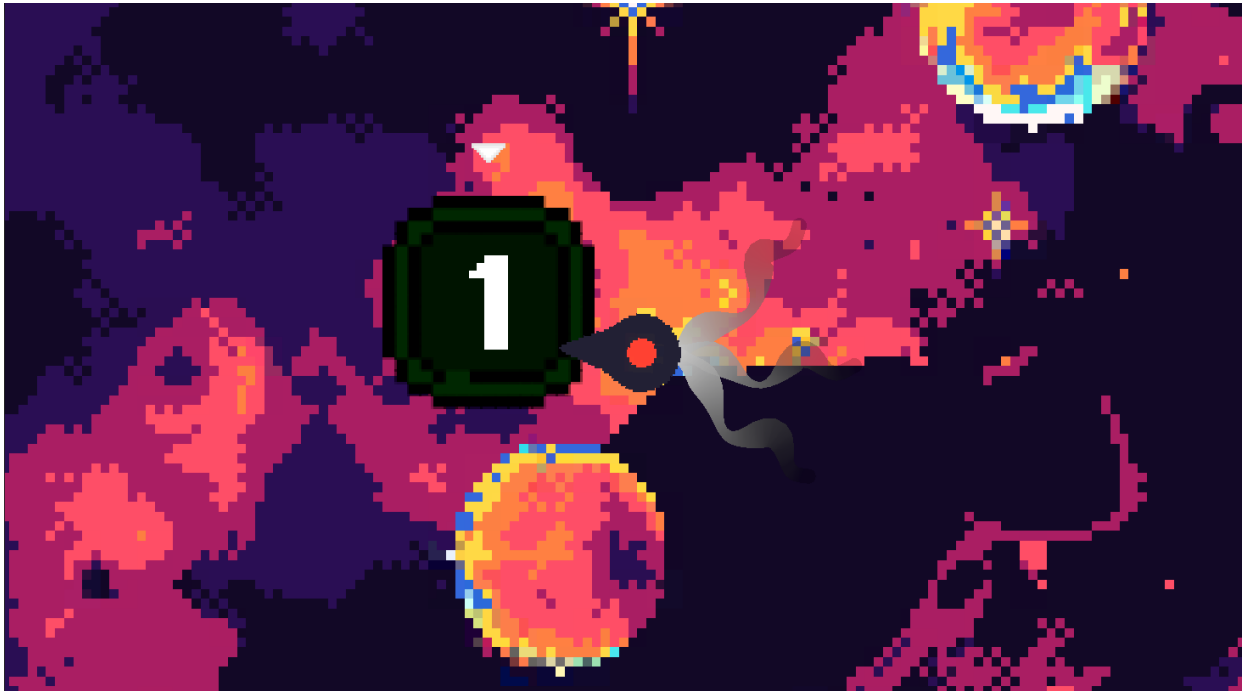


Figure 16 Hub scene interface

- **Character:** The player-controlled character, which can move freely within the hub area.

- **Level Boxes:** Interactive objects that allow the player to enter different level scenes. Each level is only unlocked after meeting specific progression conditions. Locked levels are indicated with a lock icon displayed on the corresponding box. Locked levels are indicated with a lock icon displayed on the corresponding box.

3.5.3. Gameplay Scene



Figure 17 Play scene interface

- **Enemy Counter:** The game tracks the number of enemies defeated by the player. Once the player eliminates 10 enemies, the boss will appear.
- **Minimap:** Located in the upper-right corner, the minimap shows the positions of the player and enemies, using different colors (e.g., green for the player and other colors for enemies) for easy identification.
- **Pause / Game Over UI:**
 - The Pause UI allows players to temporarily stop the game, access settings, or resume gameplay.

- The Game Over UI appears when the player is defeated, providing options such as retrying the level or returning to the hub scene.
- **Guide Button:** A button that opens the in-game guide, providing instructions and gameplay tips for the player.
- **UI (Health Bar & EXP Bar):**
 - The Health Bar displays the player's remaining health.
 - The EXP Bar shows the player's experience progress, allowing them to track leveling and upgrades.

3.6. Audio Design

3.6.1. *Background Music*

Background music is designed to match the fast-paced action gameplay and space environment. Different levels and boss battles use different music tracks to create atmosphere and tension.

3.6.2. *Sound Effects*

Sound effects are used for:

- Shooting
- Explosions
- Enemy attacks
- UI interactions
- Power-up collection

These effects improve gameplay feedback and immersion.

3.7. Chapter Conclusion

This chapter presented the overall game design of *Galaxy Savior*, including gameplay mechanics, progression systems, level structure, character design, user interface, and

system analysis. The chapter also described the technical and artistic design choices used to create a challenging and engaging bullet hell game experience.

CHAPTER 4: IMPLEMENTATION AND ACHIEVED RESULTS

4.1. Achieved Results

After the implementation process, the project has achieved the following results:

- **Core gameplay completion:** Developed the main gameplay systems including player movement (mouse-based control), shooting mechanics, bullet hell system, enemy waves, and boss battles.
- **Combat and progression system:** Implemented multiple bullet types, skill system, and upgrade mechanics, allowing players to improve damage and health through level progression.
- **Level and storyline system:** Built a semi-nonlinear progression system where completing Level 1 unlocks Levels 2, 3, and 4, and completing any two of them unlocks Level 5, supporting an open-ended storyline.
- **Graphics and user interface:** Designed main interfaces such as main menu, level selection, and in-game HUD (health, score, etc.). The 2D graphics are simple, clear, and suitable for gameplay.

4.2. Setup Instructions

- ❖ Hardware requirements:
 - CPU: 1.0GHz
 - RAM: 512MB
- ❖ Software requirements:
 - Operating system: Windows 7 or later
- ❖ Installation guide:
 - Run file *application*

4.3. Result Evaluation

Advantages:

- **Smooth and responsive gameplay:** The control system based on mouse movement allows precise navigation, especially suitable for bullet hell mechanics.
- **Challenging combat system:** Diverse bullet types and complex patterns create engaging and skill-based gameplay.
- **Flexible progression:** The semi-nonlinear level system increases replayability and allows players to choose their own path.

Disadvantages:

- **Enemy AI is still simple:** Enemy behaviors and attack patterns can be further improved to increase difficulty and variety.
- **Limited content:** The number of levels and gameplay duration are still limited, which may reduce long-term engagement.

Development Direction:

- **Expand game content:** Add more levels, enemy types, and boss designs to increase gameplay depth.
- **Improve AI and mechanics:** Enhance enemy behavior and optimize bullet patterns for better challenge and balance.
- **Add new features:** Develop additional systems such as more skills, upgrades, and visual effects.
- **Potential online features:** In the future, online features such as leaderboards or multiplayer modes can be considered to increase interaction and competitiveness.

CONCLUSION

During the implementation of the topic, because I had to approach many new fields, I encountered many difficulties, however, with the help of experienced people and my own efforts, the project was completed on time and achieved promising results. Besides, due to the limited implementation time, some parts of the project have not been fully achieved as expected. In the future, the topic will continue to be edited and expanded to be more complete.

Through the process of implementing the project, I have gained valuable experience in developing applications on Unity. This knowledge will be valuable luggage in my future work.

REFERENCES

<https://docs.unity.com/>

<https://itch.io/game-assets/free>